Modelo Relacional

RELAÇÕES, ATRIBUTOS, DOMÍNIOS, TUPLAS, CHAVES E INTEGRIDADE

O modelo relacional organiza os dados por meio de relações, normalmente visualizadas como tabelas. Nesta aula, vamos compreender os elementos que formam essas relações, como atributos, domínios e tuplas, e veremos por que chaves e restrições de integridade são essenciais para manter os dados corretos, consistentes e relacionados.



Objetivos de aprendizagem

O QUE O ESTUDANTE DEVERÁ DOMINAR

Ao final, o estudante deverá identificar relações, atributos, domínios e tuplas; diferenciar esquema de instância; reconhecer superchaves, chaves candidatas, primárias, alternativas, compostas e estrangeiras; e aplicar restrições de domínio, entidade e integridade referencial. Também deverá relacionar esses conceitos às declarações SQL usadas na criação de tabelas.

- Relação
- Atributo
- Tupla
- Domínio
- Chaves
- Integridade

Por que o modelo relacional?

SIMPLICIDADE LÓGICA E INDEPENDÊNCIA DOS DADOS

Antes do modelo relacional, estruturas hierárquicas e em rede exigiam que aplicações conhecessem caminhos de navegação. A proposta relacional separou a visão lógica dos dados de detalhes físicos de armazenamento. Isso favorece consultas declarativas, manutenção, evolução do esquema e maior clareza na representação de entidades e relacionamentos do domínio.

CONCEITUAL

ALUNO — pertence — CURSO



RELACIONAL

CURSO(id PK, nome)
ALUNO(id PK, nome, curso_id FK)

A origem: E. F. Codd

O ARTIGO QUE FUNDAMENTOU O MODELO RELACIONAL

Em 1970, Edgar F. Codd publicou “A Relational Model of Data for Large Shared Data Banks”. A proposta representava dados por relações n-árias e buscava proteger usuários e aplicações de mudanças na representação interna. A ideia estabeleceu uma base matemática para organização, consistência, consulta e evolução dos bancos de dados relacionais.

1970

A Relational Model of Data for Large Shared Data Banks

E. F. Codd • IBM

independência lógica
relações n-árias

IBM Research — artigo de Codd

O que é uma relação?

A ESTRUTURA FUNDAMENTAL DO MODELO

Uma relação pode ser entendida como um conjunto de tuplas que compartilham a mesma estrutura de atributos. Em um SGBD relacional, ela é usualmente apresentada como uma tabela. Cada coluna representa um atributo, cada linha representa uma tupla, e o conjunto dessas linhas descreve ocorrências válidas de um mesmo tipo de informação.

id	nome	curso
1	Ana	SI
2	Beto	EC
3	Caio	SI

RELAÇÃO = tabela lógica

colunas → atributos
linhas → tuplas

Esquema e instância

ESTRUTURA DEFINIDA × CONTEÚDO EM UM MOMENTO

O esquema descreve a estrutura relativamente estável da relação: nome, atributos, domínios e restrições. A instância corresponde ao conjunto de tuplas existente em determinado momento. Assim, ALUNO(id, nome, curso_id) pode permanecer igual durante meses, enquanto seus registros mudam continuamente com inserções, alterações e exclusões.

ESQUEMA

ALUNO(id, nome, curso_id)



INSTÂNCIA

(1, Ana, 10)
(2, Beto, 20)

Atributos

AS PROPRIEDADES REPRESENTADAS PELAS COLUNAS

Atributo é uma propriedade relevante que descreve as tuplas de uma relação. Em ALUNO, exemplos são id, nome, data_nascimento e email. Cada atributo possui nome e domínio definidos. Um bom projeto seleciona atributos realmente necessários ao negócio, evita informações redundantes e utiliza nomes claros, estáveis e coerentes com o significado dos dados.

atributo	domínio
id	inteiro
nome	texto
nascimento	data
email	texto

Domínios

O CONJUNTO DE VALORES PERMITIDOS

O domínio define quais valores são semanticamente aceitáveis para um atributo. Ele envolve tipo, formato e regras do negócio. Uma nota pode pertencer ao domínio de números entre 0 e 10; uma UF pode aceitar siglas específicas. Tipos SQL ajudam, mas restrições como CHECK tornam o domínio mais próximo da regra real.



Tuplas

AS OCORRÊNCIAS DE UMA RELAÇÃO

Tupla é um conjunto ordenado de valores que representa uma ocorrência da relação. Em uma tabela ALUNO, cada linha descreve um aluno específico. Todos os valores da tupla devem respeitar os domínios de seus atributos e as restrições definidas. A identificação correta de cada tupla será garantida pelas chaves.

id	nome	curso
101	Ana	SI
102	Beto	EC
103	Caio	SI

uma linha = uma tupla

Grau e cardinalidade da relação

QUANTIDADE DE ATRIBUTOS E QUANTIDADE DE TUPLAS

O grau de uma relação é o número de atributos de seu esquema. A cardinalidade, neste contexto, é a quantidade de tuplas presentes em uma instância. Uma relação ALUNO com quatro atributos possui grau 4; se contém três registros naquele momento, sua cardinalidade é 3. Não confunda com cardinalidade de relacionamentos do modelo ER.

RELAÇÃO ALUNO

grau = 4 atributos

id	nome	curso	turno
1	Ana	SI	N
2	Beto	EC	M
3	Caio	SI	N

cardinalidade = 3 tuplas

Propriedades importantes

COMO PENSAR UMA RELAÇÃO DE FORMA CORRETA

Em uma relação, as tuplas representam ocorrências distintas, os atributos têm nomes definidos e cada célula deve conter um valor compatível com seu domínio. A ordem das linhas não possui significado lógico, e a ordem das colunas também não deve ser usada para atribuir semântica. O significado vem do esquema e de seus nomes.

id	nome	curso
1	Ana	SI
2	Beto	EC
3	Caio	SI

RELAÇÃO = tabela lógica

colunas → atributos
linhas → tuplas

NULL: ausência de um valor

DESCONHECIDO, NÃO INFORMADO OU NÃO APLICÁVEL

NULL representa a ausência de um valor conhecido; não significa zero, texto vazio ou espaço em branco. Seu uso exige cuidado porque comparações com NULL seguem lógica de três valores. Em projetos introdutórios, defina NOT NULL sempre que o atributo for obrigatório e documente claramente quando a ausência de informação é realmente permitida.

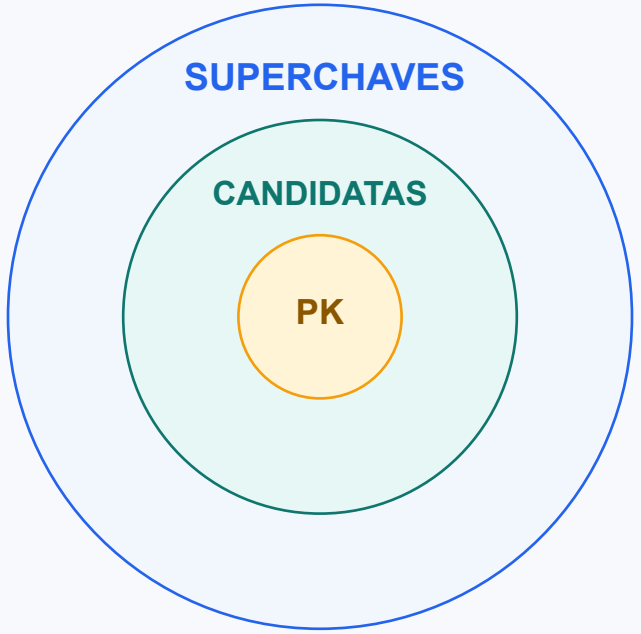
id	nome	telefone
1	Ana	27 9999
2	Beto	NULL
3	Caio	27 8888

NULL ≠ 0 ≠ texto vazio

Chaves: visão geral

COMO IDENTIFICAR E RELACIONAR TUPLAS

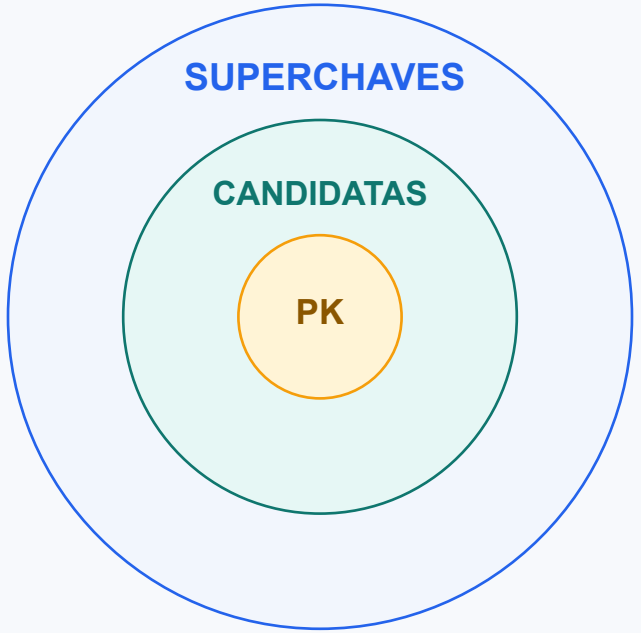
Chaves são atributos, ou conjuntos de atributos, usados para distinguir tuplas e conectar relações. A terminologia inclui superchave, chave candidata, chave primária, chave alternativa, chave composta e chave estrangeira. Cada conceito responde a uma pergunta diferente: o que identifica unicamente, qual identificador foi escolhido e como uma relação referencia outra.



Superchave

QUALQUER CONJUNTO QUE IDENTIFICA UNICAMENTE

Superchave é qualquer conjunto de atributos capaz de identificar cada tupla sem ambiguidade. Ela pode conter atributos desnecessários. Se id já identifica um aluno, então {id}, {id, nome} e {id, email} também podem ser superchaves. O objetivo seguinte é encontrar superchaves mínimas, chamadas chaves candidatas.



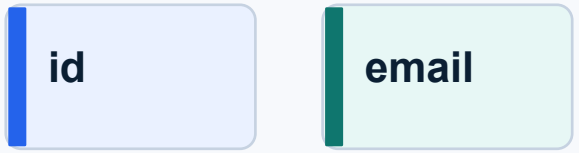
Chaves candidatas

SUPERCHAVES MÍNIMAS

Uma chave candidata identifica unicamente cada tupla e não possui atributo supérfluo. Uma relação pode ter várias candidatas. Em USUARIO, id e email podem ser únicos e, portanto, candidatos a identificador. A escolha da chave primária considera estabilidade, simplicidade, tamanho, significado no negócio e facilidade de referência por outras relações.

id	nome	email
101	Ana	ana@x.br
102	Beto	beto@x.br
103	Caio	caio@x.br

candidatas



Chave primária

A CANDIDATA ESCOLHIDA COMO IDENTIFICADOR PRINCIPAL

A chave primária é a chave candidata escolhida para identificar as tuplas da relação. Seus valores devem ser únicos e não nulos. Em SQL, PRIMARY KEY expressa essas condições e documenta o identificador oficial da tabela. Uma tabela possui no máximo uma chave primária, embora essa chave possa ser formada por várias colunas.

id	nome	email
101	Ana	ana@x.br
102	Beto	beto@x.br
103	Caio	caio@x.br

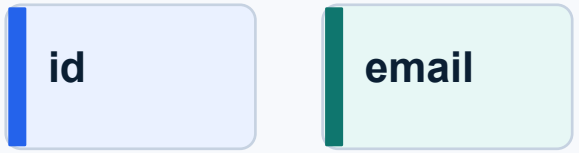
PK

Chave alternativa e chave substituta

OUTROS IDENTIFICADORES ÚTEIS

Chave alternativa é uma chave candidata que não foi escolhida como primária e pode ser protegida por UNIQUE. Já a chave substituta, ou surrogate key, é um identificador artificial, como um id numérico gerado pelo sistema. Ela simplifica referências, mas não elimina a necessidade de preservar regras de unicidade naturais do negócio quando existirem.

id	nome	email
101	Ana	ana@x.br
102	Beto	beto@x.br
103	Caio	caio@x.br



Chave composta

IDENTIFICAÇÃO USANDO MAIS DE UM ATRIBUTO

Uma chave composta utiliza duas ou mais colunas para identificar uma tupla. Em MATRICULA, a combinação aluno_id e disciplina_id pode impedir que o mesmo aluno seja matriculado duas vezes na mesma disciplina. Individualmente, os valores se repetem; em conjunto, formam uma identificação única e expressam uma regra importante do domínio.

MATRÍCULA

aluno_id + disciplina_id



juntos identificam uma única matrícula

Chave estrangeira

A REFERÊNCIA ENTRE RELAÇÕES

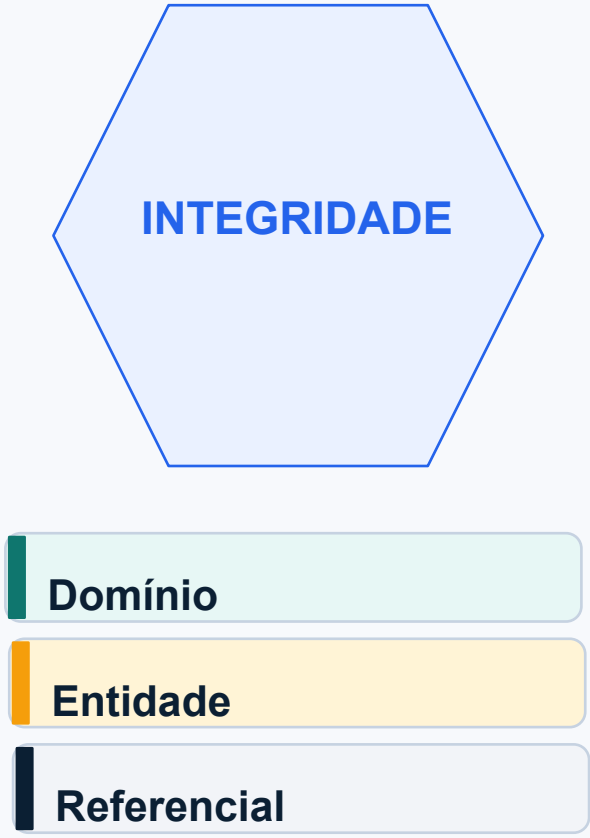
A chave estrangeira é um atributo, ou conjunto de atributos, cujo valor deve corresponder a uma chave válida da relação referenciada, salvo casos permitidos de NULL. Ela materializa relacionamentos entre tabelas e sustenta a integridade referencial. Em ALUNO, curso_id pode referenciar CURSO(id), garantindo que o curso informado realmente exista.



Integridade de dados

REGRAS QUE PRESERVAM ESTADOS VÁLIDOS

Integridade significa manter os dados coerentes com as regras definidas para o banco. No modelo relacional, é útil organizar o tema em integridade de domínio, integridade de entidade e integridade referencial. Constraints transferem parte dessas regras para o SGBD, reduzindo a dependência de validações espalhadas somente nas aplicações.



Integridade de domínio

CADA ATRIBUTO DEVE ACEITAR VALORES VÁLIDOS

A integridade de domínio exige que valores armazenados pertençam ao conjunto permitido para o atributo. Ela pode envolver tipo de dado, tamanho, formato, faixa e regras específicas. Exemplos: nota entre 0 e 10, quantidade não negativa e status restrito a valores previstos. CHECK, NOT NULL e tipos adequados auxiliam essa proteção.



Integridade de entidade

TODA TUPLA PRECISA SER IDENTIFICÁVEL

A integridade de entidade garante que cada tupla possa ser distinguida de todas as demais. A chave primária deve ser única e não nula; caso contrário, não existiria um identificador confiável para a ocorrência. Essa regra facilita atualizações, exclusões, referências por outras tabelas e rastreabilidade das informações ao longo do ciclo de vida do sistema.

id	nome	email
101	Ana	ana@x.br
102	Beto	beto@x.br
103	Caio	caio@x.br

PK

Integridade referencial

REFERÊNCIAS DEVEM APONTAR PARA DADOS EXISTENTES

A integridade referencial garante que uma chave estrangeira não aponte para uma tupla inexistente. Se ALUNO.curso_id referencia CURSO.id, inserir curso_id 99 só será permitido se o curso 99 existir, salvo NULL quando a regra aceitar ausência de vínculo. Isso evita registros órfãos e mantém relacionamentos semanticamente consistentes.



Constraints fundamentais

NOT NULL, UNIQUE, CHECK E DEFAULT

Além de PRIMARY KEY e FOREIGN KEY, SQL oferece restrições úteis para expressar regras do modelo. NOT NULL torna um valor obrigatório; UNIQUE impede duplicidades conforme a semântica do SGBD; CHECK valida uma expressão booleana; DEFAULT fornece um valor quando a coluna não é informada. Essas restrições complementam a definição de domínios e chaves.



PostgreSQL 18 — Constraints

Ações referenciais

O QUE ACONTECE AO ATUALIZAR OU EXCLUIR UMA REFERÊNCIA

Uma FOREIGN KEY pode definir ações para UPDATE e DELETE. NO ACTION ou RESTRICT normalmente impedem a operação que quebraria a referência. CASCADE propaga a alteração ou exclusão aos registros dependentes. SET NULL remove o vínculo preservando a linha, desde que a coluna aceite NULL. A escolha deve refletir a regra do negócio.

ON DELETE

NO ACTION / RESTRICT

ON DELETE CASCADE

remove dependentes

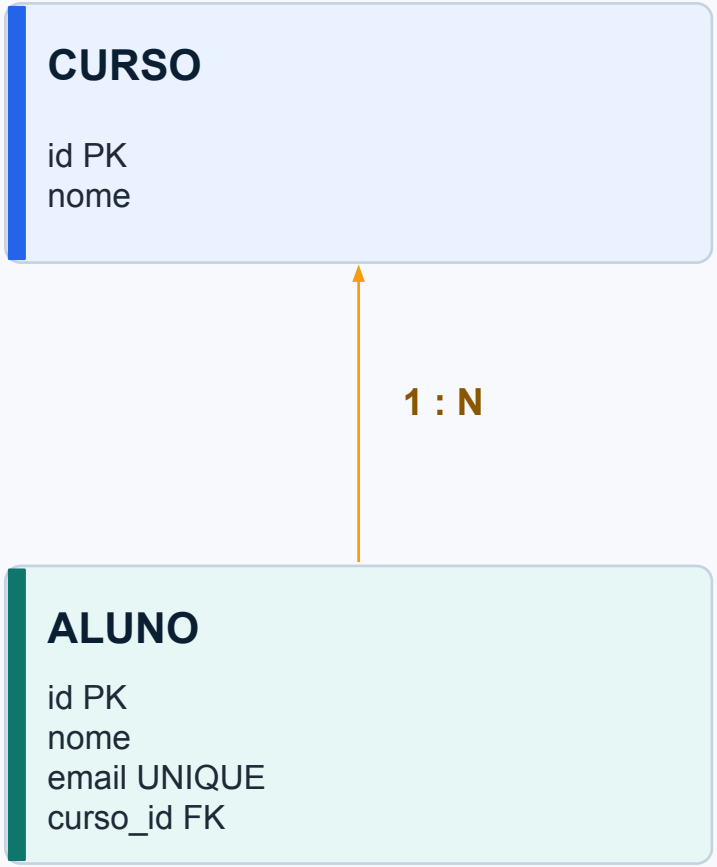
ON DELETE SET NULL

mantém linha, remove vínculo

Um caso integrado: Curso e Aluno

DO CONCEITO ÀS RELAÇÕES

Considere uma instituição em que cada aluno pertence a um curso. CURSO possui id e nome; ALUNO possui id, nome, email e curso_id. A relação 1:N é implementada com a chave estrangeira curso_id em ALUNO. Podemos ainda exigir email único, nome obrigatório e referência válida para o curso cadastrado.



Do modelo conceitual ao relacional

MAPEAMENTO DE ENTIDADES E RELACIONAMENTOS

No mapeamento básico, entidades tornam-se relações, atributos tornam-se colunas e identificadores tornam-se chaves. Em relacionamentos 1:N, a chave do lado 1 costuma migrar como chave estrangeira para o lado N. Relacionamentos M:N geralmente originam uma nova relação associativa, cujas chaves estrangeiras conectam as entidades participantes e podem formar chave composta.

CONCEITUAL

ALUNO — pertence — CURSO



RELACIONAL

CURSO(id PK, nome)
ALUNO(id PK, nome, curso_id FK)

DDL materializa o esquema

COMO OS CONCEITOS APARECEM EM SQL

Data Definition Language é a parte de SQL usada para definir e alterar estruturas do banco. CREATE TABLE descreve relações, atributos, tipos e restrições; ALTER TABLE modifica o esquema; DROP remove objetos. Assim, DDL não substitui o modelo relacional: ela é um mecanismo concreto para implementar, em um SGBD, decisões tomadas durante a modelagem.

DDL

```
CREATE TABLE aluno (
  id INTEGER PRIMARY KEY,
  nome VARCHAR(120) NOT NULL
);
```

CREATE TABLE com integridade

EXEMPLO COMPLETO EM SQL

O script ao lado implementa CURSO e ALUNO com chaves e restrições. A chave primária identifica cada linha; UNIQUE protege o email; NOT NULL torna dados obrigatórios; CHECK restringe o domínio; FOREIGN KEY garante referência válida. Observe como uma pequena definição de tabela concentra várias decisões que antes foram descritas conceitualmente.

SQL

```
CREATE TABLE curso (
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL UNIQUE
);

CREATE TABLE aluno (
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  email TEXT UNIQUE,
  idade INTEGER CHECK (idade >=
16),
  curso_id INTEGER NOT NULL
    REFERENCES curso(id)
);
```

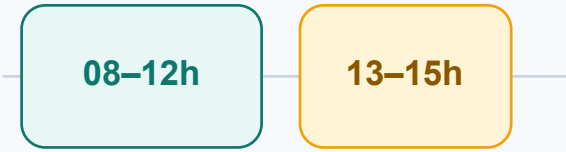
Atualização contemporânea

POSTGRESQL 18 E RESTRIÇÕES TEMPORAIS

SGBDs relacionais continuam evoluindo sem abandonar os fundamentos do modelo. O PostgreSQL 18 acrescentou recursos para restrições sobre intervalos temporais, incluindo PRIMARY KEY, UNIQUE e FOREIGN KEY com semântica de não sobreposição. Para iniciantes, a mensagem principal é simples: os mesmos princípios de chave e integridade continuam sendo ampliados para novos tipos de regra.

POSTGRESQL 18

restrições temporais



não sobrepor períodos

PostgreSQL 18 — novidades

Prática 1 — preparar a ferramenta

DBEAVER COMMUNITY COM SQLITE

Use o DBeaver Community, gratuito e de código aberto, com um banco SQLite local. Instale o DBeaver, crie uma nova conexão SQLite e selecione um arquivo de banco, por exemplo aula_relacional.db. Para esta prática, o SQLite evita configurar um servidor e permite concentrar a aula em relações, chaves, constraints e testes de integridade.

DBeaver Community + SQLite

1. **Nova conexão → SQLite**
2. SQL Editor → novo script
3. Executar CREATE TABLE
4. Testar INSERT / DELETE
5. Inspecionar dados e chaves

DBeaver Community

Prática 2 — abrir o SQL Editor

CRIAR UM SCRIPT PARA O ESQUEMA

Com a conexão aberta, clique com o botão direito no banco e escolha SQL Editor > New SQL Script. Antes de criar tabelas, execute PRAGMA foreign_keys = ON; para ativar a verificação de chaves estrangeiras no SQLite. Em seguida, escreva primeiro CURSO e depois ALUNO, respeitando a dependência criada pela FOREIGN KEY.

DBeaver Community + SQLite

1. Nova conexão → SQLite
2. **SQL Editor → novo script**
3. Executar CREATE TABLE
4. Testar INSERT / DELETE
5. Inspecionar dados e chaves

SQLite — Foreign Keys

Prática 3 — criar as relações

EXECUTAR DDL E VERIFICAR A ESTRUTURA

Crie CURSO com id INTEGER PRIMARY KEY e nome TEXT NOT NULL UNIQUE. Depois crie ALUNO com id INTEGER PRIMARY KEY, nome TEXT NOT NULL, email TEXT UNIQUE, idade INTEGER CHECK(idade >= 16) e curso_id INTEGER NOT NULL REFERENCES curso(id). Execute o script e atualize a árvore de objetos para visualizar as tabelas criadas.

DBeaver Community + SQLite

1. Nova conexão → SQLite
2. SQL Editor → novo script
3. **Executar CREATE TABLE**
4. Testar INSERT / DELETE
5. Inspecionar dados e chaves

Prática 4 — inserir dados válidos

TESTAR UMA INSTÂNCIA CORRETA

Insira primeiro dois cursos e depois alguns alunos referenciando seus ids. Verifique os resultados com SELECT * FROM curso; e SELECT * FROM aluno;. Observe que o esquema permaneceu fixo enquanto a instância mudou com cada INSERT. Confirme também que emails não se repetem e que todos os alunos apontam para cursos existentes.

DBeaver Community + SQLite

1. Nova conexão → SQLite
2. SQL Editor → novo script
3. Executar CREATE TABLE
4. **Testar INSERT / DELETE**
5. Inspecionar dados e chaves

Prática 5 — provocar violações

APRENDER OBSERVANDO O SGBD REJEITAR ERROS

Agora tente inserir um aluno com idade 12, repetir um email já cadastrado e usar curso_id 99. Cada operação deve falhar por uma razão diferente: CHECK, UNIQUE e FOREIGN KEY. Em seguida, tente excluir um curso ainda referenciado. Analise as mensagens do SGBD e identifique qual regra do modelo relacional foi protegida em cada caso.

DBeaver Community + SQLite

1. Nova conexão → SQLite
2. SQL Editor → novo script
3. Executar CREATE TABLE
4. **Testar INSERT / DELETE**
5. Inspecionar dados e chaves

Prática 6 — consultar e inspecionar vínculos

VISUALIZAR RELAÇÕES POR MEIO DE JOIN

Finalize usando uma consulta com JOIN para exibir aluno e curso na mesma resposta. O JOIN não altera a estrutura das tabelas; ele combina tuplas relacionadas pela condição ALUNO.curso_id = CURSO.id. Esse exercício reforça que as relações permanecem separadas para organização e integridade, mas podem ser recombinadas quando uma consulta precisa apresentar informações integradas.

DBeaver Community + SQLite

1. Nova conexão → SQLite
2. SQL Editor → novo script
3. Executar CREATE TABLE
4. Testar INSERT / DELETE
5. **Inspecionar dados e chaves**

Erros frequentes de modelagem

O QUE EVITAR DESDE OS PRIMEIROS PROJETOS

Alguns erros comuns são usar nome como chave sem garantir unicidade; criar uma chave artificial e esquecer regras naturais de UNIQUE; armazenar tudo como texto; permitir NULL sem necessidade; duplicar dados de outra tabela em vez de usar referência; e definir CASCADE sem compreender seus efeitos. Bons esquemas tornam as regras explícitas e verificáveis.

Nome como PK
pode se repetir

FK sem regra
órfãos

Tudo VARCHAR
domínios fracos

NULL indiscriminado
semântica confusa

Exercício orientado

MODELE UMA BIBLIOTECA ACADÊMICA

Projete relações para usuários, títulos, exemplares e empréstimos. Para cada relação, indique atributos, domínio aproximado, chave primária e possíveis chaves candidatas. Defina as chaves estrangeiras necessárias e pelo menos três restrições de integridade. Depois justifique como o modelo impediria empréstimo de exemplar inexistente, duplicidade indevida e registros sem identificação.

DESAFIO

Biblioteca acadêmica

LIVRO

isbn ?
título

EMPRÉSTIMO

? PK
? FK

USUÁRIO

id
nome

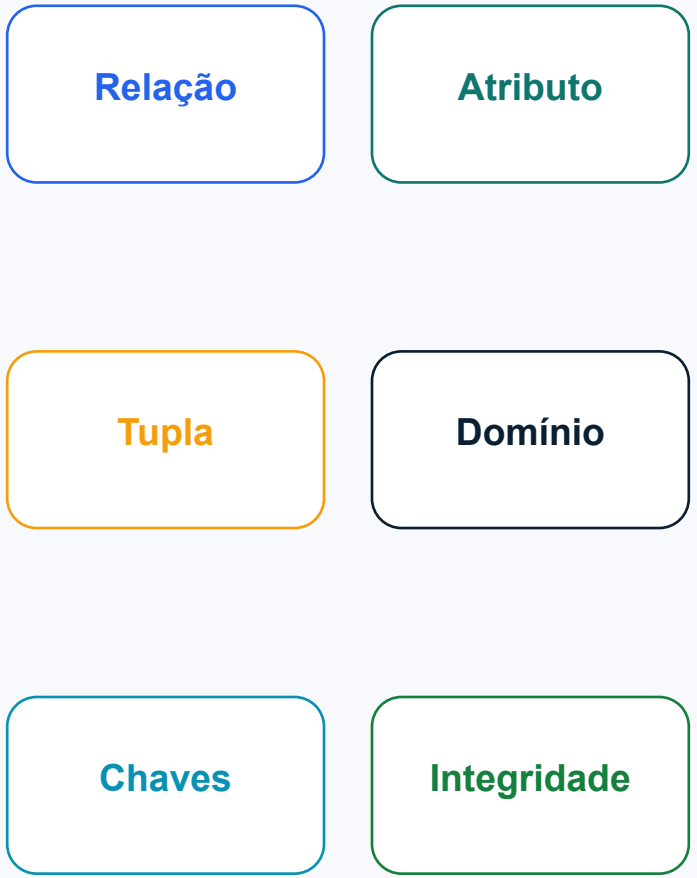
EXEMPLAR

id
livro_id

Questões para revisão

CHEQUE SUA COMPREENSÃO ANTES DE AVANÇAR

Explique a diferença entre atributo e domínio; entre esquema e instância; entre superchave e chave candidata; e entre chave primária e chave estrangeira. Depois responda: por que PRIMARY KEY contribui para a integridade de entidade? Como uma FOREIGN KEY protege a integridade referencial? Em quais casos CHECK é melhor do que confiar apenas no tipo da coluna?



Síntese da aula

OS SEIS CONCEITOS QUE ESTRUTURAM O TEMA

Relações organizam conjuntos de tuplas; atributos descrevem propriedades; domínios limitam valores válidos; chaves identificam e conectam registros; e restrições de integridade impedem estados inconsistentes. Quando esses elementos são bem definidos, o banco se torna mais confiável, compreensível e fácil de consultar, evoluir e integrar com diferentes aplicações.

- Relação
- Atributo
- Tupla
- Domínio
- Chaves
- Integridade

Recursos para continuar estudando

DOCUMENTAÇÃO E VÍDEO DE APOIO

Para aprofundar, consulte a documentação de constraints do PostgreSQL 18, a documentação do DBeaver Community e o guia de chaves estrangeiras do SQLite. Como demonstração prática adicional, o canal oficial do DBeaver possui um vídeo sobre CREATE TABLE. Esses recursos permitem revisar conceitos e repetir os exercícios em um ambiente gratuito de aprendizagem.

POSTGRESQL 18

Constraints

DBEAVER COMMUNITY

Ferramenta gratuita

VÍDEO

CREATE TABLE — DBeaver

Vídeo: Learn SQL — CREATE TABLE (DBeaver)

Referências

MATERIAIS UTILIZADOS NA ELABORAÇÃO DA AULA

A aula foi construída a partir do material fornecido pelo docente, da apostila Banco de Dados I de Simone M. V. Romano e de referências atuais e históricas: Codd (1970), documentação PostgreSQL 18, DBeaver Community e SQLite. Para aprofundamento bibliográfico, também é indicada a obra Banco de dados, de Pichetti, Vida e Cortes, citada no material anexado.

IBM Research — Codd (1970)

PostgreSQL 18 — Constraints

DBeaver Community

SQLite — Foreign Keys

BASES UTILIZADAS

Material anexado
Apostila Banco de Dados I
Codd (1970)
PostgreSQL 18
DBeaver Community
SQLite Docs

consultadas em ago./2026